Advanced SQL Assignment

Name - Gagan Namdev

Steps Performed:

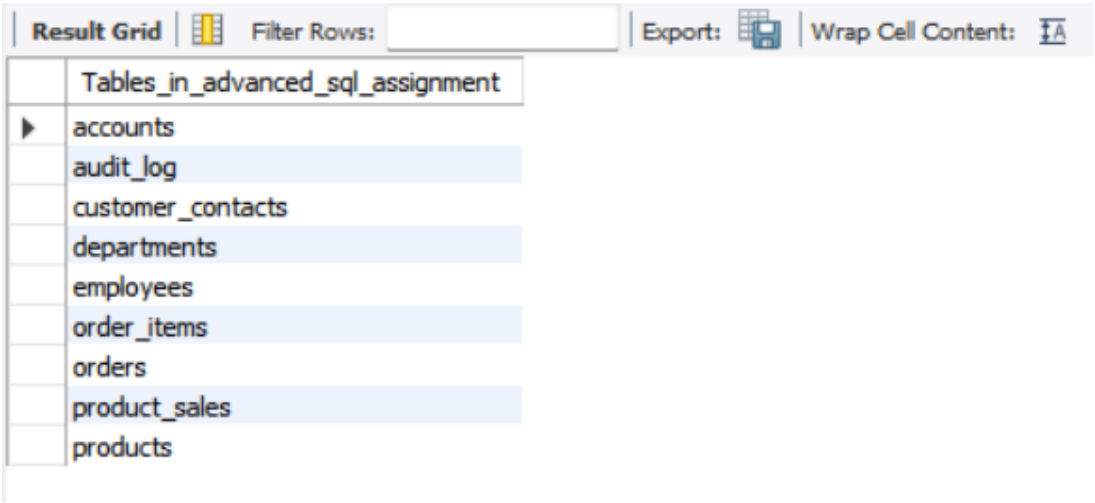
- **Schema Definition:** Identified key entities, selected appropriate data types (`INT`, `VARCHAR`, `DATE`), and applied rules like `NOT NULL` or `DEFAULT`.
- **Primary & Foreign Keys:** Established relational integrity to connect tables logically and avoid data redundancy.
- **Table Generation:** Executed Data Definition Language (DDL) scripts to create the structural tables.

```
1
2   DROP DATABASE IF EXISTS advanced_sql_assignment;
3   CREATE DATABASE advanced_sql_assignment;
4
5   USE advanced_sql_assignment;
6
7   CREATE TABLE departments (
8       department_id INT PRIMARY KEY,
9       department_name VARCHAR(50) NOT NULL
10  );
11
12  CREATE TABLE employees (
13      employee_id INT PRIMARY KEY,
14      employee_name VARCHAR(100) NOT NULL,
15      department_id INT,
16      salary DECIMAL(10,2) NOT NULL,
17      hire_date DATE NOT NULL,
18      manager_id INT NULL,
19      email VARCHAR(120) NULL,
20      mobile_phone VARCHAR(20) NULL,
21      office_phone VARCHAR(20) NULL,
22      CONSTRAINT fk_employee_department
23          FOREIGN KEY (department_id)
24          REFERENCES departments(department_id)
25  );
```

```sql

CREATE TABLE customer_contacts (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
    office_phone VARCHAR(20) NULL,
    mobile_phone VARCHAR(20) NULL,
    email VARCHAR(120) NULL
);

CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL
);

CREATE TABLE product_sales (
    sale_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    total_revenue DECIMAL(12,2) NOT NULL,
    units_sold INT NOT NULL
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    order_date DATE NOT NULL,
    delivery_date DATE NULL,
    order_status VARCHAR(20) NOT NULL,
    total_amount DECIMAL(12,2) NOT NULL,
    CONSTRAINT fk_orders_customer
        FOREIGN KEY (customer_id)
        REFERENCES customer_contacts(customer_id)
);

CREATE TABLE order_items (
    order_item_id INT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL,
    CONSTRAINT fk_order_items_order
        FOREIGN KEY (order_id)
        REFERENCES orders(order_id),
    CONSTRAINT fk_order_items_product
        FOREIGN KEY (product_id)
        REFERENCES products(product_id)
);

```

```sql
CREATE TABLE accounts (
    account_id VARCHAR(10) PRIMARY KEY,
    account_name VARCHAR(100) NOT NULL,
    balance DECIMAL(12,2) NOT NULL
);

CREATE TABLE audit_log (
    audit_id INT PRIMARY KEY AUTO_INCREMENT,
    event VARCHAR(100) NOT NULL,
    event_time DATETIME NOT NULL
);

show tables;
```

These are all the tables in our database

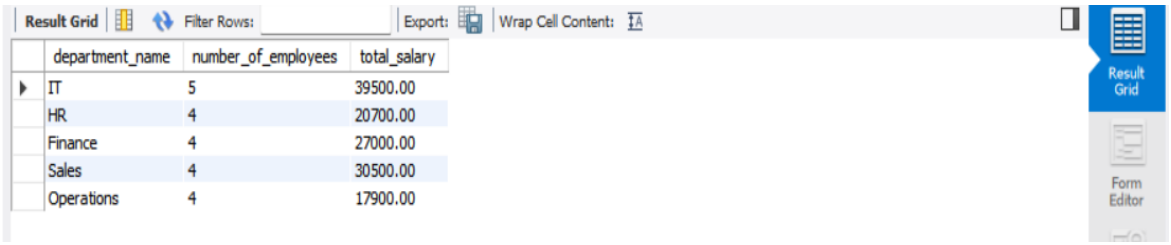


Q #	Topic	Questions
1	JOIN + Aggregation	Display each department's department_name, number of employees and total salary. Use JOIN and GROUP BY.

Query:

```
107     -- Q.1
108 •   SELECT
109         d.department_name,
110         COUNT(e.employee_id) AS number_of_employees,
111         SUM(e.salary) AS total_salary
112     FROM departments d
113     INNER JOIN employees e
114         ON d.department_id = e.department_id
115     GROUP BY d.department_id, d.department_name;
```

OUTPUT



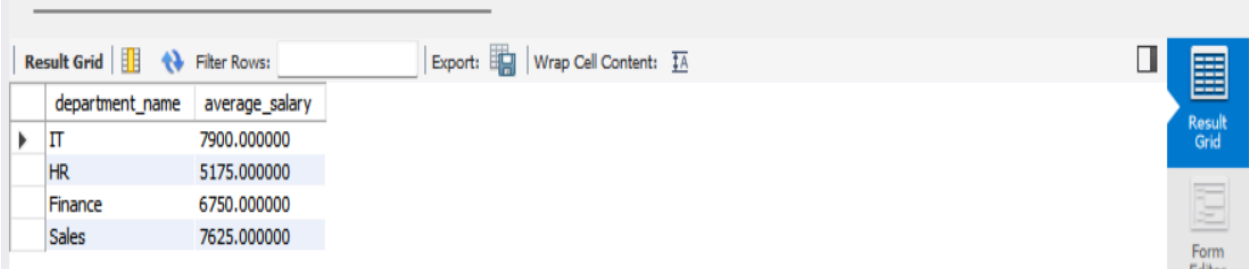
department_name	number_of_employees	total_salary
IT	5	39500.00
HR	4	20700.00
Finance	4	27000.00
Sales	4	30500.00
Operations	4	17900.00

2	JOIN + Aggregation	Display each department's department_name and average employee salary. Show only departments where the average salary is greater than 5000.

Query:

```
•   SELECT
        d.department_name,
        AVG(e.salary) AS average_salary
    FROM departments d
    INNER JOIN employees e
        ON d.department_id = e.department_id
    GROUP BY d.department_id, d.department_name
    HAVING AVG(e.salary) > 5000;
```

OUTPUT :

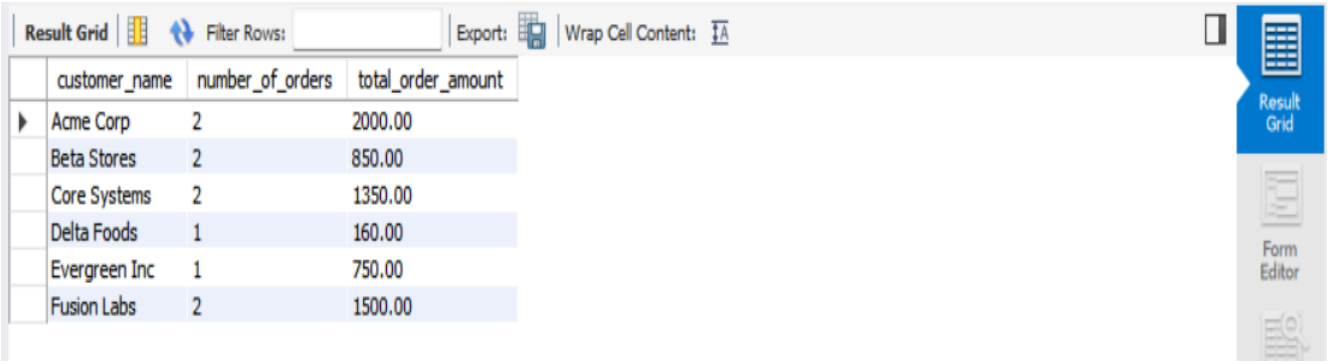


3	JOIN + Aggregation	For each customer, display customer_name, number of orders and total order amount. Include customers who have no orders.

Query:

```
SELECT
    c.customer_name,
    COUNT(o.order_id) AS number_of_orders,
    COALESCE(SUM(o.total_amount), 0.00) AS total_order_amount
FROM customer_contacts c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name;
```

OUTPUT:

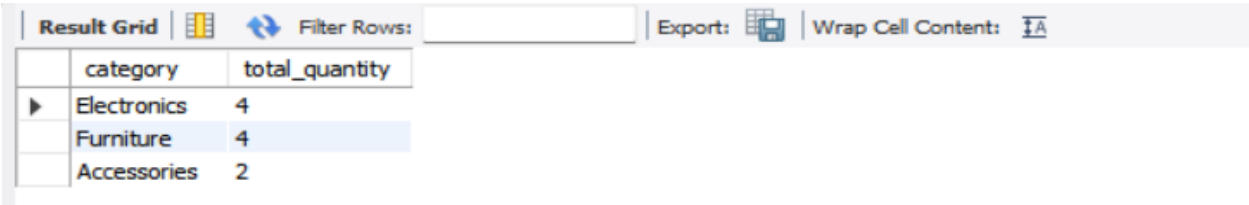


4	JOIN + Aggregation	For each product category, calculate the total quantity sold using products and order_items. Display category and total_quantity.

Query:

```
SELECT
    p.category,
    SUM(oi.quantity) AS total_quantity
FROM products p
INNER JOIN order_items oi
    ON p.product_id = oi.product_id
GROUP BY p.category;
```

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

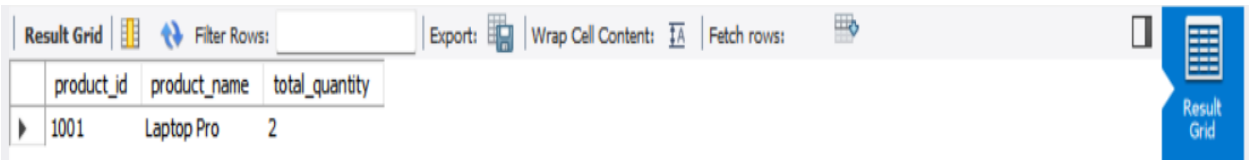
category	total_quantity
Electronics	4
Furniture	4
Accessories	2

5	JOIN + Aggregation	Find the product with the highest total quantity sold. Display product_id, product_name and total_quantity.

Query:

```
SELECT
    p.product_id,
    p.product_name,
    SUM(oi.quantity) AS total_quantity
FROM products p
INNER JOIN order_items oi
    ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name
ORDER BY total_quantity DESC
LIMIT 1;
```

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

product_id	product_name	total_quantity
1001	Laptop Pro	2

6	RANK vs DENSE_RANK	Display employee_id, salary, RANK() and DENSE_RANK() over all employees. Explain the difference when salaries are tied.

Query:

```
SELECT
    employee_id,
    salary,
    RANK() OVER (ORDER BY salary DESC) AS salary_rank,
    DENSE_RANK() OVER (ORDER BY salary DESC) AS salary_dense_rank
FROM employees;
```

OUTPUT:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	salary	salary_rank	salary_dense_rank
401	9500.00	1	1
101	9000.00	2	2
402	9000.00	2	2
301	8500.00	4	3
102	8000.00	5	4
103	8000.00	5	4
104	7500.00	7	5
105	7000.00	8	6
302	7000.00	8	6
303	7000.00	8	6
201	6500.00	11	7
403	6000.00	12	8
404	6000.00	12	8
501	5500.00	14	9
202	5000.00	15	10
203	5000.00	15	10
502	4800.00	17	11
503	4800.00	17	11
304	4500.00	19	12
204	4200.00	20	13
504	2800.00	21	14

The Difference When Salaries Are Tied :

1. RANK() (Leaves Gaps)

- Skips ranks if there is a tie.
- It calculates the rank of the next row as: (Number of rows ranked before it) + 1.
- Example: If two employees tie for Rank 2, the next employee jumps straight to Rank 4 (Rank 3 is skipped).

2. DENSE_RANK() (No Gaps)

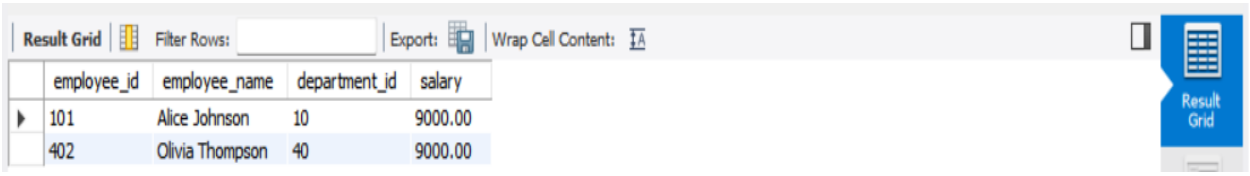
- Never skips ranks, keeping the numbering sequential ("dense").
- Example: If two employees tie for Rank 2, the next employee receives Rank 3, regardless of how many people tied before them.

7	Window Function	Using a window function, find all employees receiving the second-highest distinct salary.

Query:

```
WITH RankedEmployees AS (
    SELECT
        employee_id,
        employee_name,
        department_id,
        salary,
        DENSE_RANK() OVER (ORDER BY salary DESC) AS salary_dense_rank
    FROM employees
)
SELECT
    employee_id,
    employee_name,
    department_id,
    salary
FROM RankedEmployees
WHERE salary_dense_rank = 2;
```

OUTPUT :



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

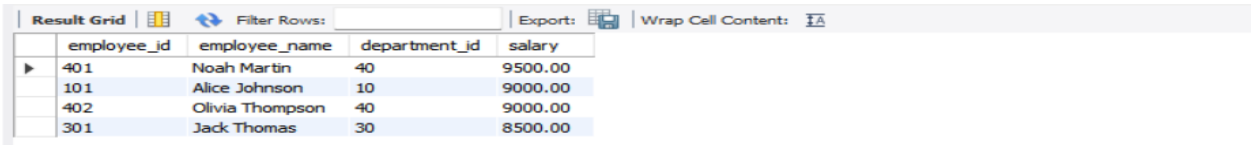
employee_id	employee_name	department_id	salary
101	Alice Johnson	10	9000.00
402	Olivia Thompson	40	9000.00

8	DENSE_RANK	Using DENSE_RANK(), return employees belonging to the top 3 distinct salary levels.

Query:

```
SELECT employee_id, employee_name, department_id, salary
FROM (
    SELECT *, DENSE_RANK() OVER (ORDER BY salary DESC) AS salary_rank
    FROM employees
) t
WHERE salary_rank <= 3;
```

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

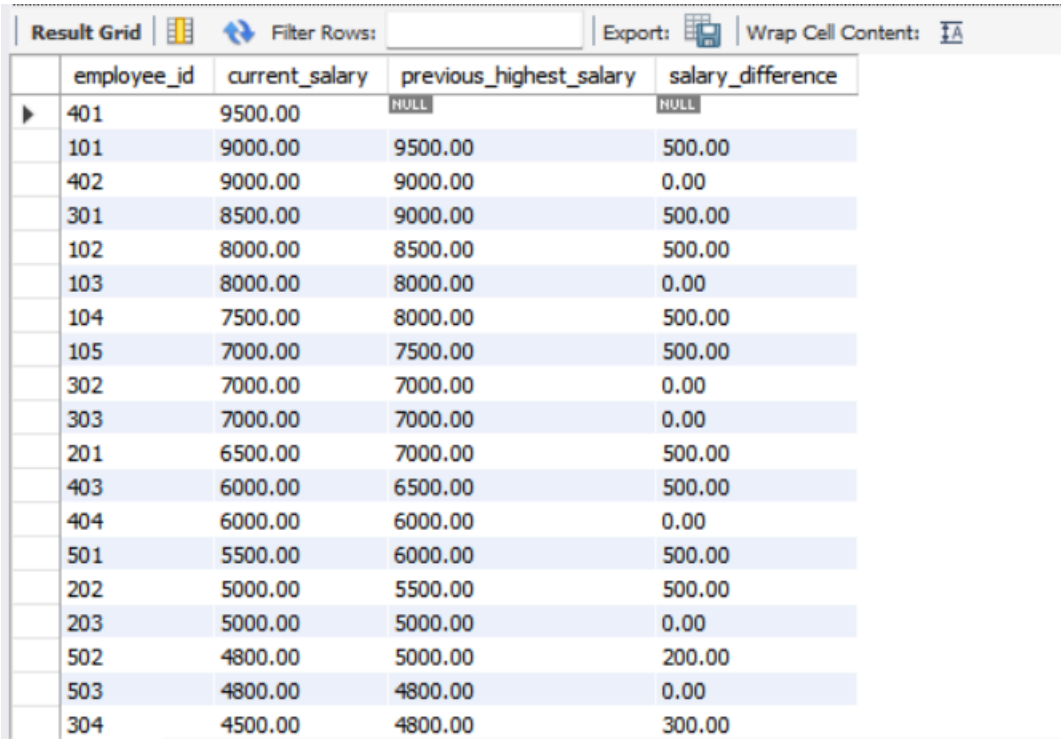
employee_id	employee_name	department_id	salary
401	Noah Martin	40	9500.00
101	Alice Johnson	10	9000.00
402	Olivia Thompson	40	9000.00
301	Jack Thomas	30	8500.00

9 . LAG	Using LAG(), display employee_id, salary, previous salary and the difference between current salary and previous salary.

Query:

```
SELECT
    employee_id,
    salary AS current_salary,
    LAG(salary) OVER (ORDER BY salary DESC) AS previous_highest_salary,
    (LAG(salary) OVER (ORDER BY salary DESC) - salary) AS salary_difference
FROM employees;
```

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

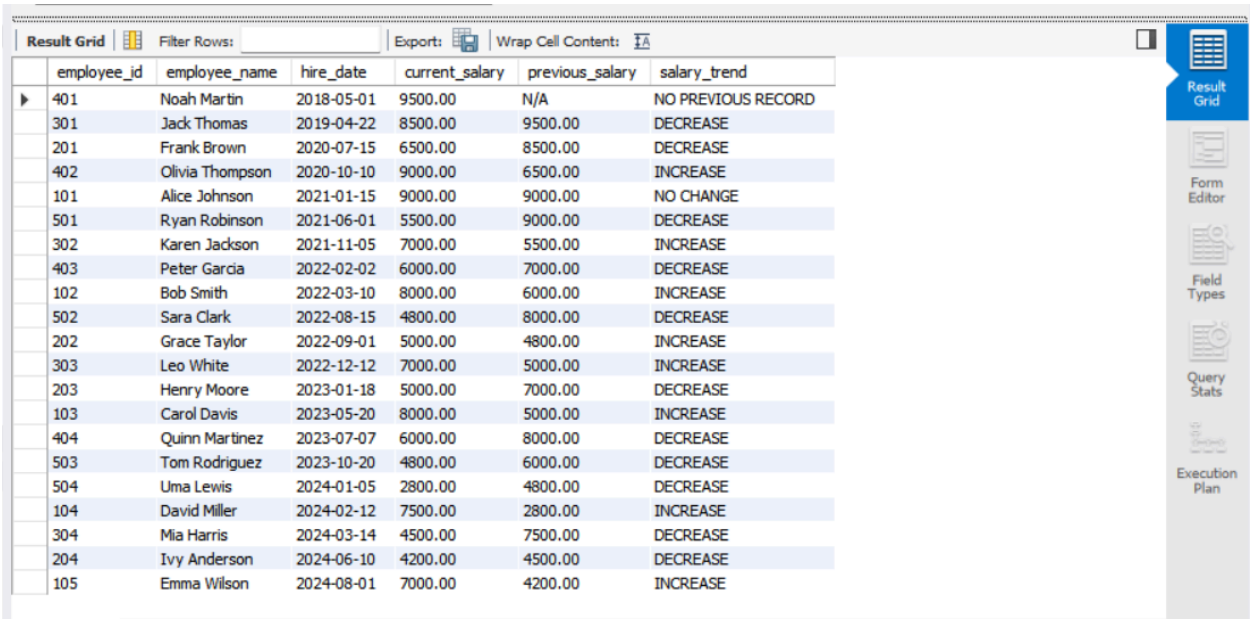
employee_id	current_salary	previous_highest_salary	salary_difference
401	9500.00	NULL	NULL
101	9000.00	9500.00	500.00
402	9000.00	9000.00	0.00
301	8500.00	9000.00	500.00
102	8000.00	8500.00	500.00
103	8000.00	8000.00	0.00
104	7500.00	8000.00	500.00
105	7000.00	7500.00	500.00
302	7000.00	7000.00	0.00
303	7000.00	7000.00	0.00
201	6500.00	7000.00	500.00
403	6000.00	6500.00	500.00
404	6000.00	6000.00	0.00
501	5500.00	6000.00	500.00
202	5000.00	5500.00	500.00
203	5000.00	5000.00	0.00
502	4800.00	5000.00	200.00
503	4800.00	4800.00	0.00
304	4500.00	4800.00	300.00

10	LAG + CASE	Using LAG() + CASE, show INCREASE, DECREASE, NO CHANGE or NO PREVIOUS RECORD.

Query:

```sql
WITH SalaryTracking AS (
    SELECT
        employee_id,
        employee_name,
        hire_date,
        salary AS current_salary,
        LAG(salary) OVER (ORDER BY hire_date ASC) AS previous_employee_salary
    FROM employees
)
SELECT
    employee_id,
    employee_name,
    hire_date,
    current_salary,
    COALESCE(CAST(previous_employee_salary AS CHAR), 'N/A') AS previous_salary,
    CASE
        WHEN previous_employee_salary IS NULL THEN 'NO PREVIOUS RECORD'
        WHEN current_salary > previous_employee_salary THEN 'INCREASE'
        WHEN current_salary < previous_employee_salary THEN 'DECREASE'
        ELSE 'NO CHANGE'
    END AS salary_trend
FROM SalaryTracking;
```

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	employee_name	hire_date	current_salary	previous_salary	salary_trend
401	Noah Martin	2018-05-01	9500.00	N/A	NO PREVIOUS RECORD
301	Jack Thomas	2019-04-22	8500.00	9500.00	DECREASE
201	Frank Brown	2020-07-15	6500.00	8500.00	DECREASE
402	Olivia Thompson	2020-10-10	9000.00	6500.00	INCREASE
101	Alice Johnson	2021-01-15	9000.00	9000.00	NO CHANGE
501	Ryan Robinson	2021-06-01	5500.00	9000.00	DECREASE
302	Karen Jackson	2021-11-05	7000.00	5500.00	INCREASE
403	Peter Garcia	2022-02-02	6000.00	7000.00	DECREASE
102	Bob Smith	2022-03-10	8000.00	6000.00	INCREASE
502	Sara Clark	2022-08-15	4800.00	8000.00	DECREASE
202	Grace Taylor	2022-09-01	5000.00	4800.00	INCREASE
303	Leo White	2022-12-12	7000.00	5000.00	INCREASE
203	Henry Moore	2023-01-18	5000.00	7000.00	DECREASE
103	Carol Davis	2023-05-20	8000.00	5000.00	INCREASE
404	Quinn Martinez	2023-07-07	6000.00	8000.00	DECREASE
503	Tom Rodriguez	2023-10-20	4800.00	6000.00	DECREASE
504	Uma Lewis	2024-01-05	2800.00	4800.00	DECREASE
104	David Miller	2024-02-12	7500.00	2800.00	INCREASE
304	Mia Harris	2024-03-14	4500.00	7500.00	DECREASE
204	Ivy Anderson	2024-06-10	4200.00	4500.00	DECREASE
105	Emma Wilson	2024-08-01	7000.00	4200.00	INCREASE

11	Running Total	Create a running total of salaries ordered by employee_id.

Query:

```
SELECT
    employee_id,
    employee_name,
    salary,
    SUM(salary) OVER (ORDER BY employee_id ASC) AS running_total_salary
FROM employees;
```

OUTPUT:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	employee_name	salary	running_total_salary
101	Alice Johnson	9000.00	9000.00
102	Bob Smith	8000.00	17000.00
103	Carol Davis	8000.00	25000.00
104	David Miller	7500.00	32500.00
105	Emma Wilson	7000.00	39500.00
201	Frank Brown	6500.00	46000.00
202	Grace Taylor	5000.00	51000.00
203	Henry Moore	5000.00	56000.00
204	Ivy Anderson	4200.00	60200.00
301	Jack Thomas	8500.00	68700.00
302	Karen Jackson	7000.00	75700.00
303	Leo White	7000.00	82700.00
304	Mia Harris	4500.00	87200.00
401	Noah Martin	9500.00	96700.00
402	Olivia Thompson	9000.00	105700.00
403	Peter Garcia	6000.00	111700.00
404	Quinn Martinez	6000.00	117700.00
501	Ryan Robinson	5500.00	123200.00
502	Sara Clark	4800.00	128000.00
503	Tom Rodriguez	4800.00	132800.00
504	Uma Lewis	2800.00	135600.00

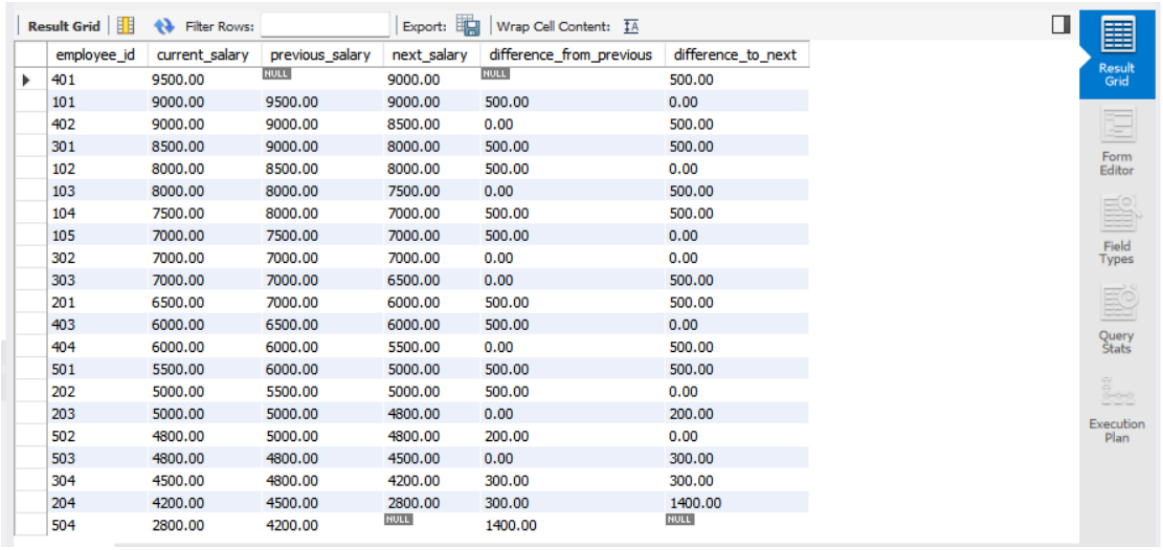
Result 23

12	LAG + LEAD	Display employee_id, salary, previous salary, next salary, difference from previous and difference to next.

Query:

```
SELECT
    employee_id,
    salary AS current_salary,
    LAG(salary) OVER (ORDER BY salary DESC) AS previous_salary,
    LEAD(salary) OVER (ORDER BY salary DESC) AS next_salary,
    (LAG(salary) OVER (ORDER BY salary DESC) - salary) AS difference_from_previous,
    (salary - LEAD(salary) OVER (ORDER BY salary DESC)) AS difference_to_next
FROM employees;
```

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	current_salary	previous_salary	next_salary	difference_from_previous	difference_to_next
401	9500.00	NULL	9000.00	NULL	500.00
101	9000.00	9500.00	9000.00	500.00	0.00
402	9000.00	9000.00	8500.00	0.00	500.00
301	8500.00	9000.00	8000.00	500.00	500.00
102	8000.00	8500.00	8000.00	500.00	0.00
103	8000.00	8000.00	7500.00	0.00	500.00
104	7500.00	8000.00	7000.00	500.00	500.00
105	7000.00	7500.00	7000.00	500.00	0.00
302	7000.00	7000.00	7000.00	0.00	0.00
303	7000.00	7000.00	6500.00	0.00	500.00
201	6500.00	7000.00	6000.00	500.00	500.00
403	6000.00	6500.00	6000.00	500.00	0.00
404	6000.00	6000.00	5500.00	0.00	500.00
501	5500.00	6000.00	5000.00	500.00	500.00
202	5000.00	5500.00	5000.00	500.00	0.00
203	5000.00	5000.00	4800.00	0.00	200.00
502	4800.00	5000.00	4800.00	200.00	0.00
503	4800.00	4800.00	4500.00	0.00	300.00
304	4500.00	4800.00	4200.00	300.00	300.00
204	4200.00	4500.00	2800.00	300.00	1400.00
504	2800.00	4200.00	NULL	1400.00	NULL

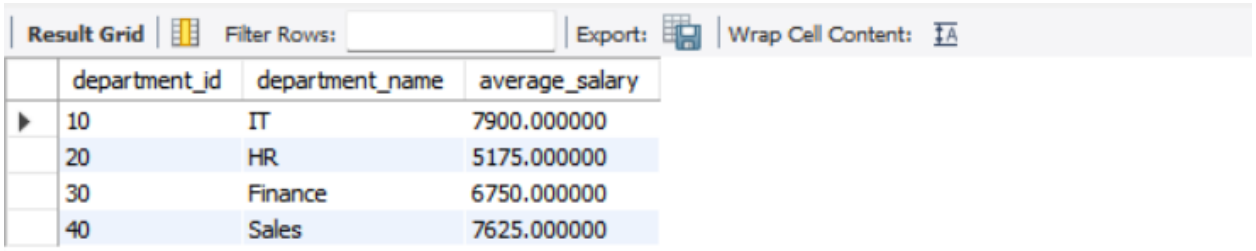
Result Grid | Form Editor | Field Types | Query Stats | Execution Plan

13.	CTE	Using a CTE, calculate average salary per department and return only departments whose average salary is greater than 5000.

Query:

```sql
WITH DepartmentAverages AS (
    SELECT
        d.department_id,
        d.department_name,
        AVG(e.salary) AS average_salary
    FROM departments d
    INNER JOIN employees e
        ON d.department_id = e.department_id
    GROUP BY d.department_id, d.department_name
)
SELECT
    department_id,
    department_name,
    average_salary
FROM DepartmentAverages
WHERE average_salary > 5000;
```

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

department_id	department_name	average_salary
10	IT	7900.000000
20	HR	5175.000000
30	Finance	6750.000000
40	Sales	7625.000000

14	CTE + JOIN	Using a CTE, return employees whose salary is greater than their own department's average salary.

Query:

```sql
WITH DepartmentAverage AS (
    SELECT
        department_id,
        AVG(salary) AS avg_dept_salary
    FROM employees
    GROUP BY department_id
)
SELECT
    e.employee_id,
    e.employee_name,
    e.department_id,
    e.salary AS employee_salary,
    ROUND(da.avg_dept_salary, 2) AS department_average_salary,
    (e.salary - da.avg_dept_salary) AS amount_above_average
FROM employees e
INNER JOIN DepartmentAverage da
    ON e.department_id = da.department_id
WHERE e.salary > da.avg_dept_salary;
```

OUTPUT:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	employee_name	department_id	employee_salary	department_average_salary	amount_above_average
101	Alice Johnson	10	9000.00	7900.00	1100.000000
102	Bob Smith	10	8000.00	7900.00	100.000000
103	Carol Davis	10	8000.00	7900.00	100.000000
201	Frank Brown	20	6500.00	5175.00	1325.000000
301	Jack Thomas	30	8500.00	6750.00	1750.000000
302	Karen Jackson	30	7000.00	6750.00	250.000000
303	Leo White	30	7000.00	6750.00	250.000000
401	Noah Martin	40	9500.00	7625.00	1875.000000
402	Olivia Thompson	40	9000.00	7625.00	1375.000000
501	Ryan Robinson	50	5500.00	4475.00	1025.000000
502	Sara Clark	50	4800.00	4475.00	325.000000
503	Tom Rodriguez	50	4800.00	4475.00	325.000000

Result Grid | Form Editor | Field Types | Query Stats

15	COALESCE	Return the first available contact in this order: office_phone -> mobile_phone -> email -> 'No Contact Info'.

Query:

```
SELECT
    customer_id,
    customer_name,
    COALESCE(office_phone, mobile_phone, email, 'No Contact Info') AS primary_contact
FROM customer_contacts;
```

OUTPUT:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	primary_contact
1	Acme Corp	555-3001
2	Beta Stores	555-3102
3	Core Systems	core@example.com
4	Delta Foods	No Contact Info
5	Evergreen Inc	555-3005
6	Fusion Labs	555-3106

16	NULLIF	Calculate revenue per unit from product_sales without dividing by zero. Use NULLIF().

Query:

```
SELECT
    sale_id,
    product_name,
    total_revenue,
    units_sold,
    total_revenue / NULLIF(units_sold, 0) AS revenue_per_unit
FROM product_sales;
```

OUTPUT:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

sale_id	product_name	total_revenue	units_sold	revenue_per_unit
1	Laptop Pro	12000.00	10	1200.000000
2	Laptop Basic	8000.00	10	800.000000
3	Monitor 27	3500.00	10	350.000000
4	Keyboard	800.00	10	80.000000
5	Mouse	400.00	10	40.000000
6	Desk	4500.00	10	450.000000
7	Office Chair	3000.00	10	300.000000
8	Premium Chair	4500.00	0	NULL

Result Grid | Form Editor | Field

17	CASE	Create salary categories and calculate bonus using CASE: >=8000 gets 15%, otherwise 10%.

Query:

```
SELECT
    employee_id,
    employee_name,
    salary,
    CASE
        WHEN salary >= 8000 THEN 'High Salary'
        ELSE 'Standard Salary'
    END AS salary_category,
    CASE
        WHEN salary >= 8000 THEN salary * 0.15
        ELSE salary * 0.10
    END AS calculated_bonus
FROM employees;
```

OUTPUT:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	employee_name	salary	salary_category	calculated_bonus
101	Alice Johnson	9000.00	High Salary	1350.0000
102	Bob Smith	8000.00	High Salary	1200.0000
103	Carol Davis	8000.00	High Salary	1200.0000
104	David Miller	7500.00	Standard Salary	750.0000
105	Emma Wilson	7000.00	Standard Salary	700.0000
201	Frank Brown	6500.00	Standard Salary	650.0000
202	Grace Taylor	5000.00	Standard Salary	500.0000
203	Henry Moore	5000.00	Standard Salary	500.0000
204	Ivy Anderson	4200.00	Standard Salary	420.0000
301	Jack Thomas	8500.00	High Salary	1275.0000
302	Karen Jackson	7000.00	Standard Salary	700.0000
303	Leo White	7000.00	Standard Salary	700.0000
304	Mia Harris	4500.00	Standard Salary	450.0000
401	Noah Martin	9500.00	High Salary	1425.0000
402	Olivia Thompson	9000.00	High Salary	1350.0000
403	Peter Garcia	6000.00	Standard Salary	600.0000
404	Quinn Martinez	6000.00	Standard Salary	600.0000
501	Ryan Robinson	5500.00	Standard Salary	550.0000
502	Sara Clark	4800.00	Standard Salary	480.0000
503	Tom Rodriguez	4800.00	Standard Salary	480.0000
504	Uma Lewis	2800.00	Standard Salary	280.0000

18	CASE + NULL	Create salary_category and contact_status using CASE and NULL checks.

Query:

```
SELECT
    employee_id,
    employee_name,
    salary,
    CASE
        WHEN salary IS NULL THEN 'Salary Missing'
        WHEN salary >= 8000 THEN 'Tier 1 (High)'
        WHEN salary >= 5000 THEN 'Tier 2 (Medium)'
        ELSE 'Tier 3 (Standard)'
    END AS salary_category,
    CASE
        WHEN mobile_phone IS NOT NULL THEN 'Mobile Available'
        WHEN office_phone IS NOT NULL THEN 'Office Only'
        ELSE 'No Phone Contact'
    END AS contact_status
FROM employees;
```

OUTPUT:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

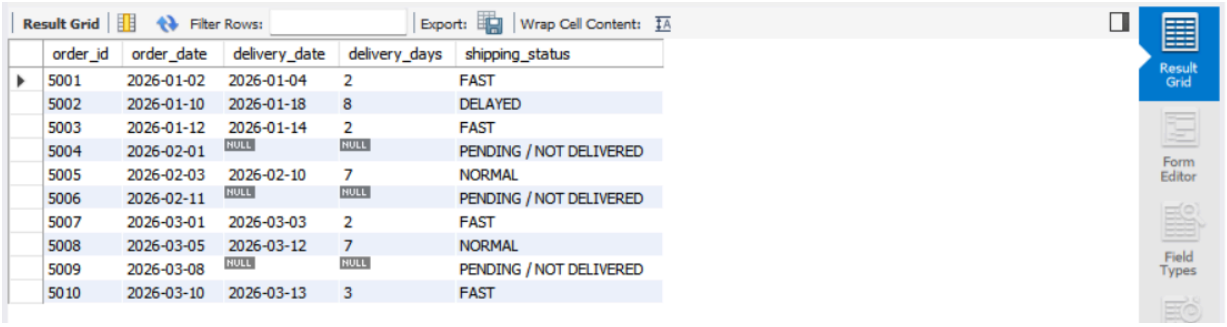
employee_id	employee_name	salary	salary_category	contact_status
101	Alice Johnson	9000.00	Tier 1 (High)	Mobile Available
102	Bob Smith	8000.00	Tier 1 (High)	Office Only
103	Carol Davis	8000.00	Tier 1 (High)	Mobile Available
104	David Miller	7500.00	Tier 2 (Medium)	Office Only
105	Emma Wilson	7000.00	Tier 2 (Medium)	Mobile Available
201	Frank Brown	6500.00	Tier 2 (Medium)	Mobile Available
202	Grace Taylor	5000.00	Tier 2 (Medium)	Office Only
203	Henry Moore	5000.00	Tier 2 (Medium)	Mobile Available
204	Ivy Anderson	4200.00	Tier 3 (Standard)	Office Only
301	Jack Thomas	8500.00	Tier 1 (High)	Mobile Available
302	Karen Jackson	7000.00	Tier 2 (Medium)	Office Only
303	Leo White	7000.00	Tier 2 (Medium)	Mobile Available
304	Mia Harris	4500.00	Tier 3 (Standard)	Office Only
401	Noah Martin	9500.00	Tier 1 (High)	Mobile Available
402	Olivia Thompson	9000.00	Tier 1 (High)	Office Only
403	Peter Garcia	6000.00	Tier 2 (Medium)	Mobile Available
404	Quinn Martinez	6000.00	Tier 2 (Medium)	Office Only
501	Ryan Robinson	5500.00	Tier 2 (Medium)	Mobile Available
502	Sara Clark	4800.00	Tier 3 (Standard)	Mobile Available
503	Tom Rodriguez	4800.00	Tier 3 (Standard)	Office Only
504	Uma Lewis	2800.00	Tier 3 (Standard)	Office Only

19	DATEDIFF + CASE	Calculate delivery days using DATEDIFF and classify <=3 FAST, 4-7 NORMAL, >7 DELAYED.

Query:

```
SELECT
    order_id,
    order_date,
    delivery_date,
    DATEDIFF(delivery_date, order_date) AS delivery_days,
    CASE
        WHEN delivery_date IS NULL THEN 'PENDING / NOT DELIVERED'
        WHEN DATEDIFF(delivery_date, order_date) <= 3 THEN 'FAST'
        WHEN DATEDIFF(delivery_date, order_date) BETWEEN 4 AND 7 THEN 'NORMAL'
        ELSE 'DELAYED'
    END AS shipping_status
FROM orders;
```

OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	order_date	delivery_date	delivery_days	shipping_status
5001	2026-01-02	2026-01-04	2	FAST
5002	2026-01-10	2026-01-18	8	DELAYED
5003	2026-01-12	2026-01-14	2	FAST
5004	2026-02-01	NULL	NULL	PENDING / NOT DELIVERED
5005	2026-02-03	2026-02-10	7	NORMAL
5006	2026-02-11	NULL	NULL	PENDING / NOT DELIVERED
5007	2026-03-01	2026-03-03	2	FAST
5008	2026-03-05	2026-03-12	7	NORMAL
5009	2026-03-08	NULL	NULL	PENDING / NOT DELIVERED
5010	2026-03-10	2026-03-13	3	FAST

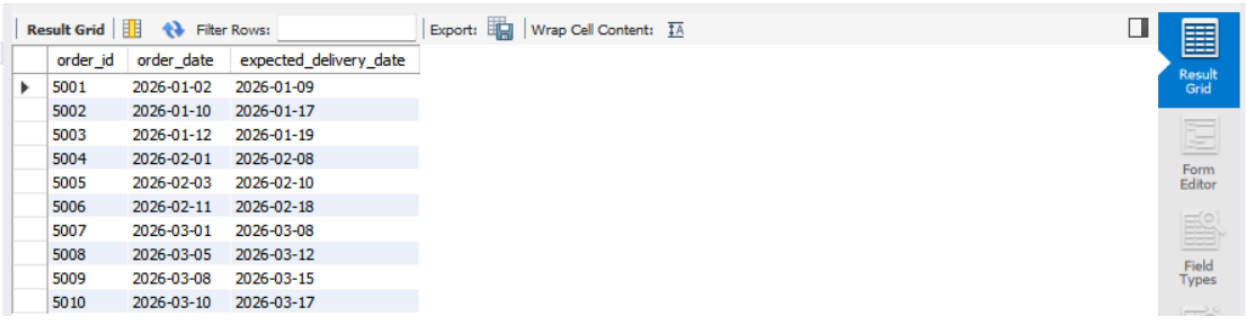
Result Grid | Form Editor | Field Types

20	DATEADD	Using DATEADD, calculate expected delivery date as order_date + 7 days.

Query:

```
SELECT
    order_id,
    order_date,
    DATE_ADD(order_date, INTERVAL 7 DAY) AS expected_delivery_date
FROM orders;
```

OUTPUT:



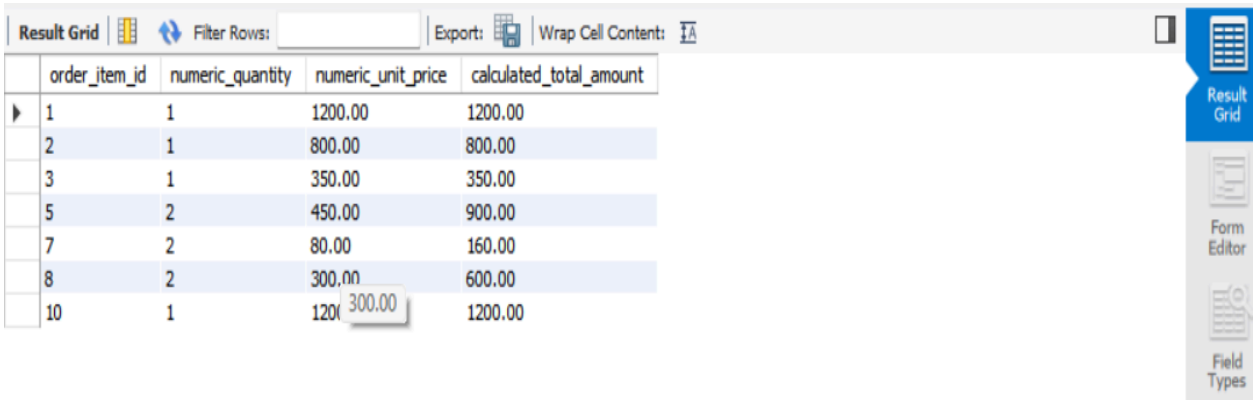
order_id	order_date	expected_delivery_date
5001	2026-01-02	2026-01-09
5002	2026-01-10	2026-01-17
5003	2026-01-12	2026-01-19
5004	2026-02-01	2026-02-08
5005	2026-02-03	2026-02-10
5006	2026-02-11	2026-02-18
5007	2026-03-01	2026-03-08
5008	2026-03-05	2026-03-12
5009	2026-03-08	2026-03-15
5010	2026-03-10	2026-03-17

21	CAST / CONVERT	Convert quantity and price from string values into numeric types and calculate total amount.

Query:

```
SELECT
    order_item_id,

    CAST(quantity AS SIGNED) AS numeric_quantity,
    CAST(unit_price AS DECIMAL(10,2)) AS numeric_unit_price,

    (CAST(quantity AS SIGNED) * CAST(unit_price AS DECIMAL(10,2))) AS calculated_total_amount
FROM order_items;
```

OUTPUT:



order_item_id	numeric_quantity	numeric_unit_price	calculated_total_amount
1	1	1200.00	1200.00
2	1	800.00	800.00
3	1	350.00	350.00
5	2	450.00	900.00
7	2	80.00	160.00
8	2	300.00	600.00
10	1	1200.00	1200.00

22	Stored Procedure	Create GetEmployeeDetails accepting @emp_id and returning employee details. Execute it for employee 101.

Query:

The DELIMITER // command in MySQL is used to temporarily change the standard statement terminator from a semicolon (;) to a different symbol (in this case, double forward slashes //) .

```
-- Q.22
DELIMITER //

CREATE PROCEDURE GetEmployeeDetails (
    IN p_emp_id INT
)
BEGIN
    SELECT
        e.employee_id,
        e.employee_name,
        d.department_name,
        e.salary,
        e.hire_date,
        e.email,
        e.mobile_phone
    FROM employees e
    LEFT JOIN departments d
        ON e.department_id = d.department_id
    WHERE e.employee_id = p_emp_id;
END //

DELIMITER ;
```

Query:

```
CALL GetEmployeeDetails(101);
```

Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	employee_name	department_name	salary	hire_date	email	mobile_phone
101	Alice Johnson	IT	9000.00	2021-01-15	alice@company.com	555-1001

23	Trigger	Create an AFTER INSERT trigger on employees that inserts 'New Employee Added' and current timestamp into audit_log.

Step 1: Create the Trigger

Query :

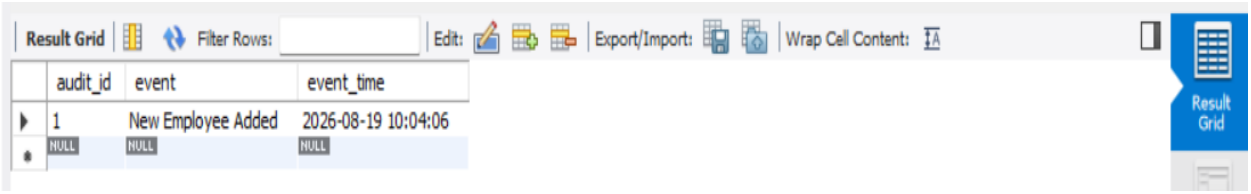
```
DELIMITER //

CREATE TRIGGER after_employee_insert
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    INSERT INTO audit_log (event, event_time)
    VALUES ('New Employee Added', NOW());
END //

DELIMITER ;
```

Step 2:Test and Verify the Trigger

Query :

```
INSERT IGNORE INTO departments (department_id, department_name)
VALUES (1, 'HR'), (2, 'IT'), (3, 'Finance');

INSERT INTO employees (employee_id, employee_name, department_id, salary, hire_date, email)
VALUES (506, 'Rahul Verma', 2, 5500.00, '2026-08-19', 'rahul@email.com');

select * from audit_log
```

Output :



24	Transaction + ACID	Transfer $500 from account A to B using B EGIN TRANSACTION, COMMIT and ROLLBACK with TRY/CATCH. Explain Atomicity.

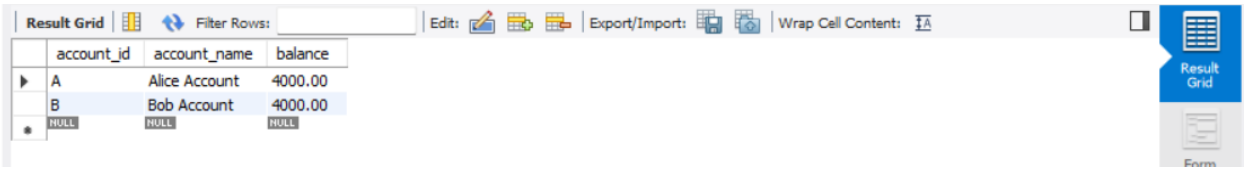
Query :

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 500.00
WHERE account_id = 'A';

UPDATE accounts
SET balance = balance + 500.00
WHERE account_id = 'B';

COMMIT;
```

Step to Verify :

```
-- verify
SELECT * FROM accounts WHERE account_id IN ('A', 'B');
```

Output:



25	Combined Challenge	Using a CTE, calculate department average salary; use DENSE_RANK() PARTITION BY department; return employees above department average; add salary_category using CASE.

Query :

```sql
WITH RankedDepartmentStaff AS (
    SELECT
        e.employee_id,
        e.employee_name,
        e.department_id,
        e.salary,
        -- 1. Calculate department average without collapsing rows
        AVG(e.salary) OVER(PARTITION BY e.department_id) AS dept_avg_salary,
        -- 2. Rank employees inside their own department based on earnings
        DENSE_RANK() OVER(PARTITION BY e.department_id ORDER BY e.salary DESC) AS dept_salary_rank
    FROM employees e
)
SELECT
    employee_id,
    employee_name,
    department_id,
    salary,
    ROUND(dept_avg_salary, 2) AS department_average,
    dept_salary_rank,
    -- 3. Categorize employee salary bands using a CASE statement
    CASE
        WHEN salary >= 8000 THEN 'Tier 1 (High Earner)'
        WHEN salary >= 5000 THEN 'Tier 2 (Mid-Level)'
        ELSE 'Tier 3 (Standard)'
    END AS salary_category
FROM RankedDepartmentStaff
-- 4. Filter to keep only employees earning strictly above their department average
WHERE salary > dept_avg_salary;
```

Output :

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	employee_name	department_id	salary	department_average	dept_salary_rank	salary_category
101	Alice Johnson	10	9000.00	7900.00	1	Tier 1 (High Earner)
102	Bob Smith	10	8000.00	7900.00	2	Tier 1 (High Earner)
103	Carol Davis	10	8000.00	7900.00	2	Tier 1 (High Earner)
201	Frank Brown	20	6500.00	5175.00	1	Tier 2 (Mid-Level)
301	Jack Thomas	30	8500.00	6750.00	1	Tier 1 (High Earner)
302	Karen Jackson	30	7000.00	6750.00	2	Tier 2 (Mid-Level)
303	Leo White	30	7000.00	6750.00	2	Tier 2 (Mid-Level)
401	Noah Martin	40	9500.00	7625.00	1	Tier 1 (High Earner)
402	Olivia Thompson	40	9000.00	7625.00	2	Tier 1 (High Earner)
501	Ryan Robinson	50	5500.00	4475.00	1	Tier 2 (Mid-Level)
502	Sara Clark	50	4800.00	4475.00	2	Tier 3 (Standard)
503	Tom Rodriguez	50	4800.00	4475.00	2	Tier 3 (Standard)